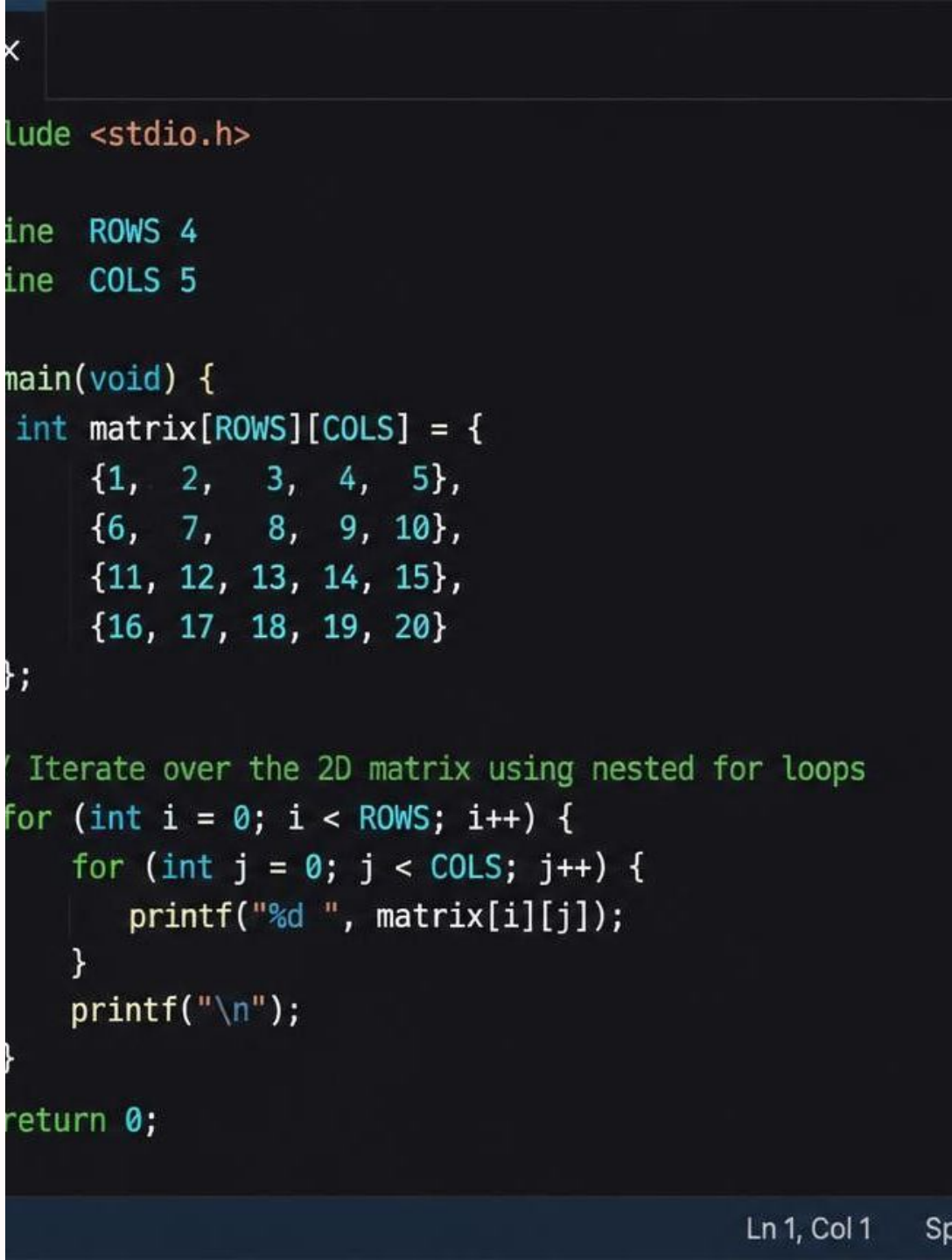
Gli Array Multidimensionali in C

Dichiarazione e inizializzazione, accesso e scorrimento, operazioni su matrici, matrici e funzioni, griglie di caratteri

LINGUAGGIO C | MATRICI | STRUTTURE DATI



Cos'è una Matrice?

Una matrice è un array di array: una tabella di elementi dello stesso tipo organizzati in righe e colonne. Ogni elemento si individua con due indici.

```
#define RIGHE   3
#define COLONNE 4

int voti[RIGHE][COLONNE];
voti[1][2] = 8;      /* riga 1, colonna 2 */
```

Elemento	Significato
primo indice	la riga, da 0 a RIGHE - 1
secondo indice	la colonna, da 0 a COLONNE - 1
numero di elementi	RIGHE × COLONNE

Esempi reali: orario scolastico (giorni × ore), posti di un cinema (file × posti), voti di una classe (studenti × materie).

Inizializzazione

```
int m[2][3] = {
   {1, 2, 3},
   {4, 5, 6}
};

int parziale[3][3] = { {1}, {0, 2} };  /* il resto vale 0 */
int zeri[4][5] = {0};                  /* tutta a zero */
int r[][3] = { {1, 2, 3}, {4, 5, 6} }; /* 2 righe dedotte */
```

- Graffe annidate: una coppia per ogni riga
- Gli elementi non indicati valgono 0
- Si può omettere solo la prima dimensione, mai le colonne

Le dimensioni si definiscono con #define: cambiando un solo valore si adatta tutto il programma.

La Matrice in Memoria: Ordine per Righe

La memoria del computer è una sequenza di celle numerate, non una tabella. Il compilatore "stende" la matrice in memoria una riga dopo l'altra: prima tutti gli elementi della riga 0, poi quelli della riga 1 e così via.

```
int m[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

Per trovare m[r][c] il compilatore calcola la posizione r × COLONNE + c. Esempio: m[1][2] → 1 × 3 + 2 = 5.

Posizione	**0**	**1**	**2**	**3**	**4**	**5**
elemento	m[0][0]	m[0][1]	m[0][2]	m[1][0]	m[1][1]	m[1][2]
valore	10	20	30	40	50	60

Per questo calcolo serve sempre il numero di colonne: è il motivo per cui le colonne non si possono mai omettere.

La Matrice in Memoria: sizeof e Matrici di Caratteri

```
int m[2][3];
sizeof(m)       /* 2 × 3 × 4 = 24 byte: tutta la matrice */
sizeof(m[0])    /* 3 × 4 = 12 byte: una riga */
sizeof(m[0][0]) /* 4 byte: un elemento */

righe = sizeof(m) / sizeof(m[0]);      /* 2 */
colonne = sizeof(m[0]) / sizeof(m[0][0]); /* 3 */
```

m[0] da solo indica l'intera riga 0, che è a sua volta un array di 3 interi.

Una matrice di char può contenere un elenco di nomi: ogni riga è un nome, terminato dal carattere '\0'.

```
char nomi[3][8] = { "Anna", "Marco", "Giulia" };
```

LEN deve contare anche il '\0': "Giulia" ha 6 lettere e occupa 7 caratteri, quindi entra in una riga da 8.

Scorrimento con Cicli Annidati

Per visitare tutti gli elementi servono due cicli: il ciclo esterno sceglie la riga, il ciclo interno percorre tutte le colonne di quella riga.

```
for (r = 0; r < RIGHE; r++) {
    for (c = 0; c < COLONNE; c++) {
        printf("%4d", m[r][c]);
    }
    printf("\n");     /* fine riga: a capo */
}
```

Con una matrice 2 × 3 l'ordine delle visite è:

r = 0	**[0][0] → [0][1] → [0][2]**	**a capo**
r = 1	[1][0] → [1][1] → [1][2]	a capo

Il ciclo interno ricomincia da c = 0 ogni volta che r aumenta: in totale si eseguono RIGHE × COLONNE visite.

Il printf("\n") sta fuori dal ciclo interno ma dentro quello esterno: viene eseguito una volta per riga.

Scorrimento per Colonne e Parziale

Scambiando i cicli si visita la matrice colonna per colonna: utile quando il dato da calcolare riguarda una colonna (es. la media di una materia).

```
for (c = 0; c < COLONNE; c++) {      /* esterno: colonne */
    for (r = 0; r < RIGHE; r++) {    /* interno: righe */
        somma += m[r][c];
    }
}
```

Errore tipico: invertire gli indici. Scrivendo m[c][r] al posto di m[r][c] si esce dalla matrice quando non è quadrata.

Scorrimento	Ciclo esterno	Ciclo interno	Esempio
per righe	r	c	stampa della tabella
per colonne	c	r	totali per materia
una riga fissa	—	c	voti di uno studente
una colonna fissa	—	r	voti di una materia

Accesso Sicuro

Il C non controlla gli indici. In una matrice int m[3][4]:

- m[1][5] non dà errore: la posizione 1 × 4 + 5 = 9 corrisponde a m[2][1], quindi si legge un altro elemento
- m[3][0] è oltre la fine della matrice: si legge o si scrive la memoria di altre variabili

Quando gli indici arrivano dall'utente o da un calcolo, si controllano prima di usarli:

```
if (r >= 0 && r < RIGHE && c >= 0 && c < COLONNE) {
    printf("%d\n", m[r][c]);
} else {
    printf("Cella inesistente\n");
}
```

L'ordine conta: prima i limiti, poi l'accesso. Con && il C si ferma alla prima condizione falsa e non legge la cella.

Lettura da Tastiera e Coordinate dell'Utente

Chi usa il programma conta da 1, mentre gli indici partono da 0: le coordinate inserite vanno convertite.

L'utente scrive	Conversione	Indice
riga 3	riga - 1	2
colonna 1	colonna - 1	0
fila C	lettera - 'A'	2

Per riempire una matrice da tastiera si usa il doppio ciclo; ogni valore si valida con un do-while che ripete la domanda finché il dato non è corretto.

```
do {
    printf("Giudice %d, concorrente %d: ", g + 1, c + 1);
    scanf("%d", &punteggi[g][c]);
} while (punteggi[g][c] < 0 || punteggi[g][c] > 10);
```

Nei messaggi si stampa indice + 1: l'utente vede "giudice 1", non "giudice 0".

Somme per Riga e per Colonna

Il problema: in una tabella di vendite (prodotti × mesi) servono il totale di ogni prodotto, il totale di ogni mese e il totale generale.

```
for (r = 0; r < RIGHE; r++) {
    totale_riga[r] = 0;
    for (c = 0; c < COLONNE; c++) {
        totale_riga[r] += m[r][c];
        totale_colonna[c] += m[r][c];  /* azzerato prima */
    }
}
```

I totali di riga sono RIGHE valori, quelli di colonna COLONNE valori: si memorizzano in due array monodimensionali.

	Gen	**Feb**	**Mar**	**Totale riga**
Prodotto A	10	20	30	60
Prodotto B	5	5	10	20
Totale colonna	15	25	40	80

Verifica di coerenza: la somma dei totali di riga e quella dei totali di colonna devono dare lo stesso totale generale (80).

Massimo e Minimo con Posizione

Il problema: non basta sapere qual è il valore più alto; spesso serve sapere dove si trova (quale studente, quale materia).

```
massimo = m[0][0];      /* si parte dal primo elemento */
riga_max = 0;
colonna_max = 0;
for (r = 0; r < RIGHE; r++) {
    for (c = 0; c < COLONNE; c++) {
        if (m[r][c] > massimo) {
            massimo = m[r][c];
            riga_max = r;
            colonna_max = c;
        }
    }
}
```

- Il valore iniziale è m[0][0], non 0: con valori tutti negativi 0 sarebbe un massimo sbagliato
- Insieme al valore si aggiornano riga e colonna
- Con il confronto > in caso di parità resta la prima posizione trovata

Per il minimo si usa lo stesso schema con il confronto <.

Ricerca di un Valore

Il problema: trovare la prima cella che contiene un valore e fermarsi appena trovata, senza usare break (che uscirebbe solo dal ciclo interno).

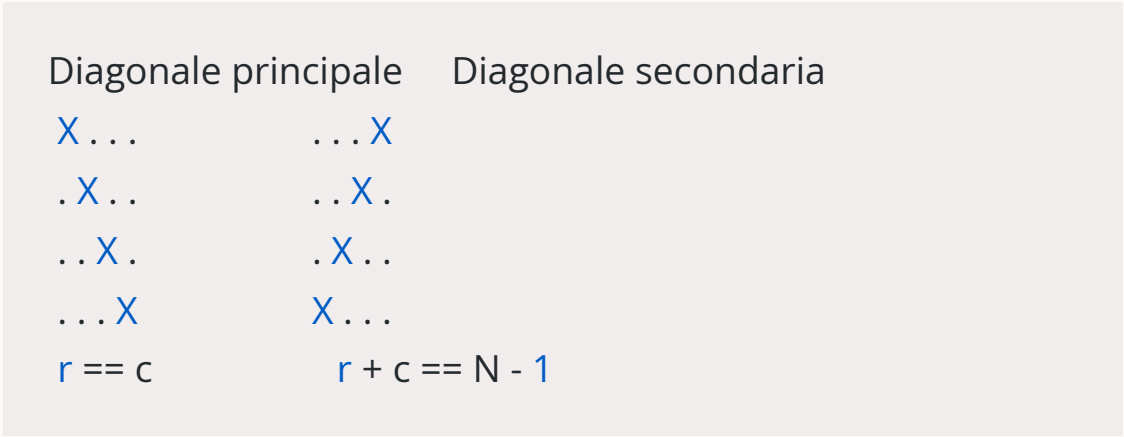
```
trovato = 0;
for (r = 0; r < RIGHE && !trovato; r++) {
    for (c = 0; c < COLONNE && !trovato; c++) {
        if (m[r][c] == cercato) {
            trovato = 1;
            riga_trovata = r;
            colonna_trovata = c;
        }
    }
}
```

- Il flag trovato compare nella condizione di entrambi i cicli: quando diventa 1 terminano tutti e due
- La posizione si salva in variabili apposite, perché all'uscita dai cicli r e c sono già stati incrementati
- Se trovato vale ancora 0, il valore non è presente

Per contare tutte le occorrenze invece si scorre tutta la matrice senza flag, incrementando un contatore.

Le Diagonali

Le diagonali esistono solo nelle matrici quadrate (N × N).



Basta un solo ciclo, perché su ogni riga c'è un solo elemento della diagonale:

```
for (i = 0; i < N; i++) {
    principale += m[i][i];
    secondaria += m[i][N - 1 - i];
}
```

Esempio d'uso: in un quadrato magico righe, colonne e le due diagonali hanno tutte la stessa somma.

La Trasposta

La trasposta scambia righe e colonne: la riga r diventa la colonna r. Una matrice RIGHE × COLONNE diventa COLONNE × RIGHE.

```
m (2 × 3)      t (3 × 2)
1 2 3          1 4
4 5 6          2 5
               3 6
```

```
for (r = 0; r < RIGHE; r++) {
    for (c = 0; c < COLONNE; c++) {
        t[c][r] = m[r][c];
    }
}
```

- Una matrice quadrata è simmetrica se coincide con la sua trasposta: m[r][c] == m[c][r] per ogni cella (es. tabella delle distanze tra città)
- Trasposta sul posto (solo matrici quadrate): si scambia m[r][c] con m[c][r] solo per c > r

Scambiando tutte le coppie, ognuna verrebbe scambiata due volte e la matrice tornerebbe com'era.

Matrici e Funzioni

Il problema: le stesse operazioni (stampa, somme, massimo) si ripetono su matrici diverse: conviene scriverle una volta in una funzione.

```
void stampa(const int m[][COLONNE], int righe);
void raddoppia(int m[][COLONNE], int righe);

stampa(voti, RIGHE);     /* nella chiamata solo il nome
*/
```

Una riga m[r] è un array: si può passare alle funzioni già scritte per gli array monodimensionali, es. somma_array(m[r], COLONNE).

- La matrice non viene copiata: la funzione lavora direttamente sui dati del chiamante e quindi può modificarli (raddoppia cambia voti)
- Il numero di colonne nel parametro è obbligatorio: serve alla funzione per calcolare la posizione r × COLONNE + c
- Il numero di righe si passa come parametro separato
- const indica che la funzione legge la matrice ma non la modifica

Funzioni su Matrici

Ogni funzione svolge un solo compito e restituisce il risultato, senza stampare.

Per restituire il massimo e la sua posizione si usano i parametri OUT del capitolo 03:

```
int cerca_massimo(const int m[][C], int n,
                  int *riga_max, int *colonna_max);

massimo = cerca_massimo(m, RIGHE, &r, &c);
```

Le matrici non si assegnano: b = a; è un errore di compilazione. Si copia elemento per elemento con una funzione copia.

Funzione	Che cosa fa
somma_riga(m, r)	scorre le colonne della riga r
somma_colonna(m, n, c)	scorre le n righe della colonna c
conta(m, n, valore)	conta le celle uguali a valore
copia(dest, sorg)	copia tutte le celle

Griglie di Caratteri: il Problema

Il problema: molti programmi devono rappresentare uno spazio diviso in caselle: la mappa di un gioco, un labirinto, la pianta di un magazzino, il campo della battaglia navale. Ogni casella ha un contenuto che va disegnato e controllato.

La soluzione: una matrice di char in cui ogni cella contiene un simbolo che descrive la casella.

Con una griglia di caratteri la mappa si stampa così com'è e il contenuto di una casella si legge con un semplice confronto: if (mappa[r][c] == '#').

Una matrice di int con codici numerici (0 = prato, 1 = acqua, ...) funzionerebbe, ma sarebbe molto meno leggibile da stampare e da scrivere.

Simbolo	Significato
.	prato o corridoio libero
^	albero
~	acqua
#	roccia o scaffale (non attraversabile)
C	casa

Griglie di Caratteri: Dichiarazione e Uso

Riempire una griglia cella per cella sarebbe lunghissimo: si scrive invece ogni riga come una stringa tra virgolette.

```
#define RIGHE   4
#define COLONNE 6

char mappa[RIGHE][COLONNE + 1] = {
    "..^^~~",
    ".^^.~~",
    "..#...",
    "C....."
};
```

- Ogni stringa termina con il carattere '\0', che occupa una cella in più: per questo la seconda dimensione è COLONNE + 1
- I cicli scorrono solo le COLONNE celle vere: c < COLONNE
- Una cella si modifica con un carattere tra apici: mappa[3][0] = '.';

Attenzione agli apici: '.' è un carattere, "." è una stringa. mappa[r][c] = "."; è un errore.

Le 8 Celle Vicine: il Problema

Il problema: nel campo minato ogni casella mostra quante mine ci sono intorno; nel gioco della vita una cella vive o muore in base a quante vicine vive ha. In entrambi i casi bisogna esaminare le 8 celle che circondano una cella [r][c].

```
[r-1][c-1]  [r-1][c]  [r-1][c+1]
[r][c-1]    [r][c]    [r][c+1]
[r+1][c-1]  [r+1][c]  [r+1][c+1]
```

Scrivere 8 if separati è lungo e facile da sbagliare. Inoltre non tutte le celle hanno 8 vicine:

Posizione della cella	Vicine reali
interna	8
sul bordo	5
in un angolo	3

Ogni vicina va controllata prima di essere letta: per una cella sul bordo alcune posizioni cadono fuori dalla griglia.

Le 8 Celle Vicine: la Soluzione con dr e dc

La soluzione: ogni vicina si ottiene sommando alla cella uno spostamento di riga dr e uno di colonna dc, che valgono -1, 0 o +1. Due cicli generano tutte le 9 combinazioni.

```
n = 0;
for (dr = -1; dr <= 1; dr++) {
   for (dc = -1; dc <= 1; dc++) {
      rv = r + dr;
      cv = c + dc;
      if (!(dr == 0 && dc == 0)
         && rv >= 0 && rv < RIGHE
         && cv >= 0 && cv < COLONNE
         && g[rv][cv] == SIMBOLO) {
         n++;
      }
   }
}
```

- La combinazione dr = 0, dc = 0 è la cella stessa e viene esclusa
- Le posizioni fuori dalla griglia vengono scartate dal controllo dei limiti
- I limiti sono scritti prima di g[rv][cv]: con && la cella fuori griglia non viene mai letta

Lo stesso schema con un solo dr, dc scelto da uno switch (N, S, E, O) sposta un personaggio sulla mappa.

Aggiornamento con Due Griglie: il Problema

Il problema: in una simulazione tutte le celle devono cambiare insieme, in base allo stato attuale. Esempio: un incendio in un bosco, con la regola "un albero ^ prende fuoco * se una cella adiacente (sopra, sotto, a sinistra o a destra) sta bruciando". A ogni passo il fuoco dovrebbe avanzare di una sola cella.

Stato iniziale: * ^ ^ ^ ^ ^

Passo 1 corretto: * * ^ ^ ^ ^

Passo 1 sul posto: * * * * * * (sbagliato)

Perché aggiornare sul posto è sbagliato: scorrendo da sinistra a destra, la cella 1 prende fuoco; quando si esamina la cella 2, la cella 1 risulta già in fiamme e anche la 2 brucia, e così via. Il fuoco attraversa tutta la riga in un solo passo.

Il risultato dipende dall'ordine di scorrimento: le celle esaminate dopo vedono valori già modificati nello stesso passo.

Aggiornamento con Due Griglie: la Soluzione

La soluzione: si usano due matrici della stessa dimensione. Si legge solo dalla griglia attuale e si scrive solo nella griglia nuova; alla fine del passo la nuova si copia nell'attuale.

```
for (r = 0; r < RIGHE; r++) {
    for (c = 0; c < COLONNE; c++) {
        if (attuale[r][c] == ALBERO
            && brucia_vicino(attuale, r, c)) {
            nuova[r][c] = FUOCO;
        } else {
            nuova[r][c] = attuale[r][c];
        }
    }
    nuova[r][COLONNE] = '\0';
}
copia(attuale, nuova);
```

Passo	Legge da	Scrive in
1. calcolo	attuale	nuova
2. copia	nuova	attuale

Griglie parallele: anche in altri problemi più matrici della stessa dimensione descrivono aspetti diversi delle stesse celle (flotta e tiri nella battaglia navale, mine e numeri nel campo minato).

Errori Comuni da Evitare

1. Invertire gli indici: m[c][r] invece di m[r][c].
2. Cicli che arrivano a RIGHE o COLONNE con <=: accesso fuori dalla matrice.
3. Leggere una cella prima di controllare i limiti.
4. Omettere il numero di colonne nel parametro: int m[][] non compila.
5. Assegnare una matrice a un'altra con b = a.
6. Dimenticare lo spazio per '\0' nelle griglie di caratteri.
7. Aggiornare una griglia mentre la si legge.
8. Usare le coordinate dell'utente senza sottrarre 1.

Riepilogo Concetti

Operazione	Tecnica	Note
Dichiarare	tipo m[RIGHE][COLONNE]	dimensioni con #define
Inizializzare	graffe annidate	{0} azzera tutto
Scorrere	due for annidati	esterno righe, interno colonne
Totali	array di riga e di colonna	verifica di coerenza
Massimo	valore + riga + colonna	ricerca fermata da flag
Diagonali	m[i][i], m[i][N-1-i]	solo matrici quadrate
Passare a funzione	int m[][COLONNE], righe	non viene copiata
Griglie di caratteri	char g[R][C + 1]	spazio per '\0'
Vicini	spostamenti dr, dc	limiti prima dell'accesso
Simulazioni	due griglie	leggi da una, scrivi nell'altra

SEZIONE 1

Esempi Pratici: Dichiarazione e Inizializzazione

Concetti base — tre file sulla creazione di una matrice.

1 – 010-dichiarazione-base.c

Problema: dichiarare e riempire una matrice.

Tecnica: dimensioni con #define, riempimento con cicli annidati, stampa in tabella.

2 – 020-inizializzazione.c

Problema: assegnare i valori iniziali.

Tecnica: graffe annidate, inizializzazione parziale, {0}, righe dedotte.

3 – 030-matrice-in-memoria.c

Problema: come è memorizzata una matrice.

Tecnica: sizeof di matrice, riga ed elemento; ordine per righe; char nomi[N] [LEN].

SEZIONE 1

Esempi Pratici: Problemi su Dichiarazione e Inizializzazione

040 – Orario scolastico

5 giorni × 6 ore con codici materia: ore per materia, giorno con più Informatica.

050 – Posti cinema

Sala 6 × 10: posti liberi per fila, riempimento, ricerca di 4 posti vicini.

060 – Temperature della settimana

7 giorni × 4 fasce con i nomi in array paralleli: medie ed estremi.

070 – Voti della classe

6 studenti × 4 materie con i nomi: medie, debiti, insufficienze per materia.

080 – Vendite trimestrali

5 prodotti × 3 mesi con codici e prezzi: totali, fatturato, prodotti in crescita.

SEZIONE 2

Esempi Pratici: Accesso e Scorrimento

Concetti base — tre file su come raggiungere le celle.

1 – 010-accesso-per-indici.c

Problema: leggere e scrivere una cella.

Tecnica: celle vicine, accesso sicuro con controllo dei limiti.

2 – 020-scorrimento-righe-e-colonne.c

Problema: visitare tutte le celle in ordini diversi.

Tecnica: per righe, per colonne, all'indietro, a serpente.

3 – 030-lettura-da-tastiera.c

Problema: riempire la matrice con l'input dell'utente.

Tecnica: punteggi di una giuria con validazione, totali per colonna.

SEZIONE 2

Esempi Pratici: Problemi su Accesso e Scorrimento

040 – Prenotazione teatro

Fila indicata con una lettera e posto: prenotazione e proposta del primo posto libero.

050 – Parcheggio multipiano

Tessere su 3 piani × 6 posti: ingresso, uscita e stato del parcheggio.

060 – Registro presenze

Matrice di char P/A/R con nomi e giorni: riepiloghi e dettaglio per studente.

070 – Stazioni meteo

Pioggia di 4 stazioni × 6 intervalli: totali, picchi e consultazione.

080 – Magazzino scaffali

Giacenze con soglia per scaffale: ripiani da rifornire e prelievi validati.

SEZIONE 3

Esempi Pratici: Operazioni su Matrici

Concetti base — tre file sui calcoli tipici.

1 – 010-somme-per-riga-e-colonna.c

Problema: totali di una tabella.

Tecnica: totali di riga, di colonna e generale; verifica di coerenza.

2 – 020-massimo-minimo-e-ricerca.c

Problema: trovare valori e posizioni.

Tecnica: estremi con posizione, ricerca con flag nei due cicli, conteggio.

3 – 030-diagonali-e-trasposta.c

Problema: operazioni sulle matrici quadrate.

Tecnica: diagonali, trasposta, simmetria, trasposta sul posto.

SEZIONE 3

Esempi Pratici: Problemi su Operazioni su Matrici

040 – Quadrato magico

Verifica di righe, colonne e diagonali sul quadrato di Dürer e su una variante.

050 – Immagine in scala di grigi

Immagine 6 × 10: visualizzazione, media, negativo, soglia, specchio.

060 – Campionato

Punti di 6 squadre in 5 giornate: classifica, imbattute, esiti per giornata.

070 – Consumi condominio

6 appartamenti in 6 mesi: totali, picco, quote e consumi elevati.

080 – Distanze tra città

Tabella simmetrica delle distanze: città più vicina, coppia più lontana, itinerario.

SEZIONE 4

Esempi Pratici: Matrici e Funzioni

Concetti base — tre file sul passaggio di matrici.

1 – 010-passaggio-matrice-a-funzione.c

Problema: stampare e modificare con una funzione.

Tecnica: parametro int m[][COLONNE], righe come parametro, una riga come array.

2 – 020-funzioni-su-matrici.c

Problema: calcoli riutilizzabili.

Tecnica: somma di riga e di colonna, conteggio, massimo con parametri OUT.

3 – 030-funzioni-che-modificano-matrici.c

Problema: trasformare una matrice.

Tecnica: riempi, copia, scala, trasposta, rotazione di 90 gradi.

SEZIONE 4

Esempi Pratici: Problemi su Matrici e Funzioni

040 – Prodotto tra matrici

Ordini × prezzi: spesa, ricavo e margine; controllo delle dimensioni.

050 – Gioco del tris

Tris per due giocatori con griglia di char e funzioni dedicate.

060 – Pagella della classe

Scrutinio: medie, esiti e statistiche per materia con funzioni di riga e colonna.

070 – Inventario negozi

4 negozi: scorta per prodotto e trasferimenti tra negozi.

080 – Turni di lavoro

5 dipendenti in 7 giorni: ore di contratto, riposi e copertura dei turni.

SEZIONE 5

Esempi Pratici: Griglie di Caratteri

Concetti base — tre file su mappe e simulazioni.

1 – 010-griglia-di-caratteri.c

Problema: rappresentare una mappa.

Tecnica: righe come stringhe, COLONNE + 1, conteggio dei simboli.

2 – 020-vicini-di-una-cella.c

Problema: esaminare le 8 celle intorno.

Tecnica: spostamenti dr, dc; mappa del rischio di un frutteto.

3 – 030-aggiornamento-in-due-griglie.c

Problema: aggiornare tutte le celle insieme.

Tecnica: incendio aggiornato sul posto (sbagliato) e con due griglie.

SEZIONE 5

Esempi Pratici: Problemi su Griglie di Caratteri

040 – Robot nel magazzino

Comandi N S E O: il robot raccoglie i pacchi evitando gli scaffali.

050 – Battaglia navale

Griglia segreta della flotta e griglia dei tiri, coordinate come C 4.

060 – Gioco della vita

Regole di Conway: aliante e lampeggiatore per 4 generazioni.

070 – Campo minato

Matrice dei numeri calcolata una volta, celle scoperte, vittoria o mina.

080 – Mappa del tesoro

Percorso guidato da due array paralleli: direzioni e passi.